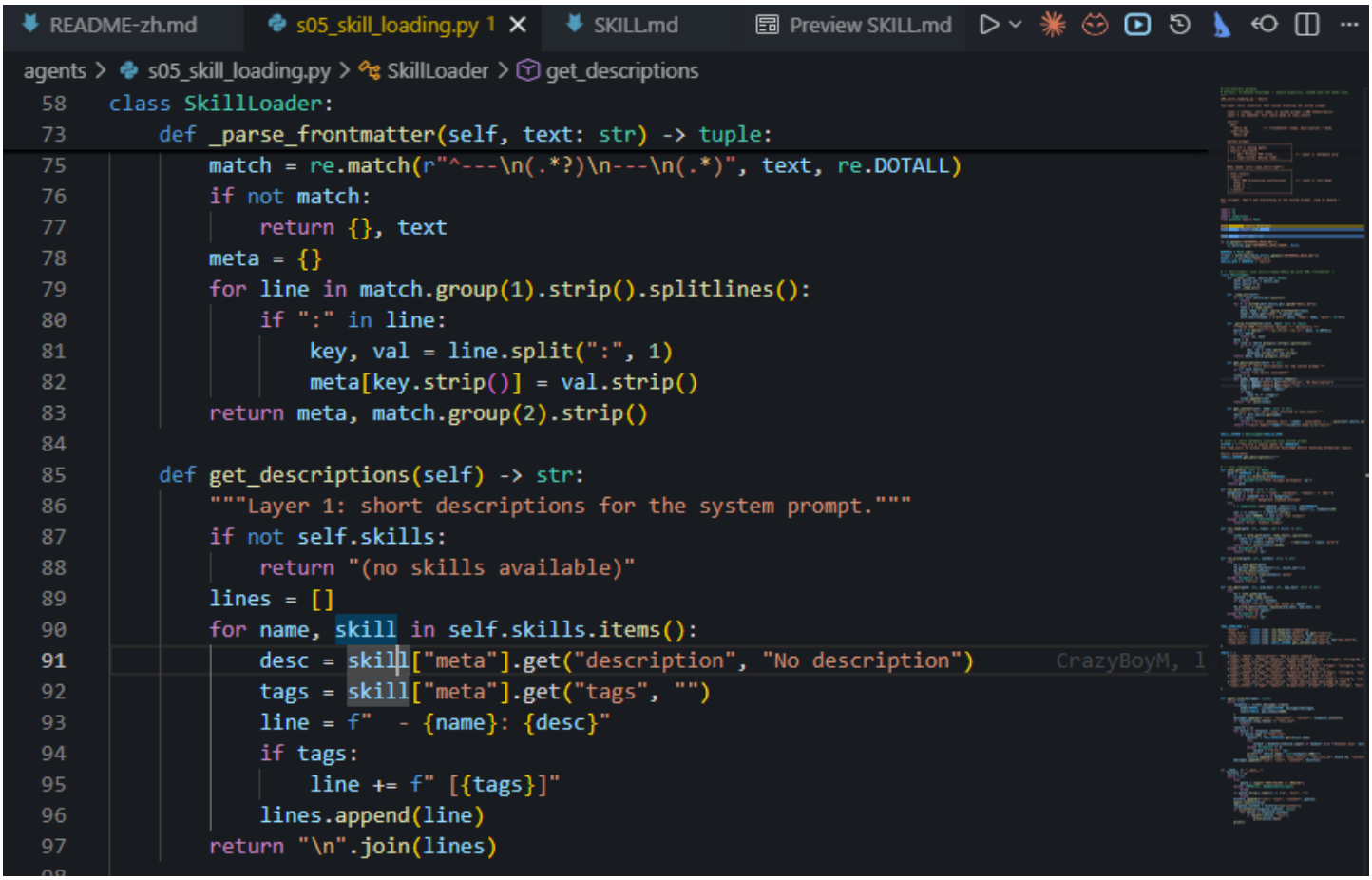
Hello Wiki项目技术文档

0. 初步了解Karpathy 的 LLM 知识库方法论

1. 从代码到知识：Andrej Karpathy 的 LLM 知识库方法论
   https://zhuanlan.zhihu.com/p/2023803203725135995

个人认为的可行性原理：

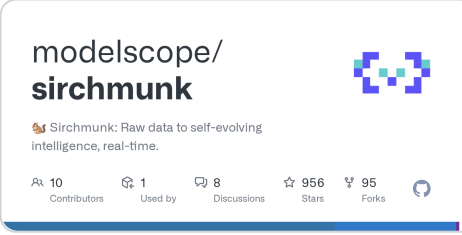
- 在诸多的实践中（如：Claude code），以meta、tag为索引文档链接子SKILL.md（我们学习手册中的文档确实没体现这一点，但是多SKILL任务是这么操作的，即当SKILL.md的内容过大时会形成一个链表，链接到子SKILL），这个链表结构也是Wiki数据库构建的底层基础。



1. 技术调研与技术选型

技术调研

参考项目：

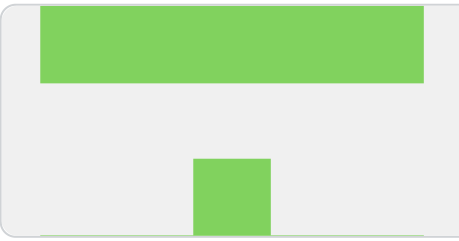


https://github.com/modelscope/sirchmunk

GitHub - modelscope/sirchmunk: 🐿️ Sirchmunk: Raw data to self-evolving intelligence, real-time.

🐿️ Sirchmunk: Raw data to self-evolving intelligence, real-time. -···

可参考项目架构与数据搜索



https://github.com/vectorize-io/hindsight

GitHub - vectorize-io/hindsight: Hindsight: Agent Memory That Learns

Hindsight: Agent Memory That Learns. Contribute to vectorize-io/hindsight···

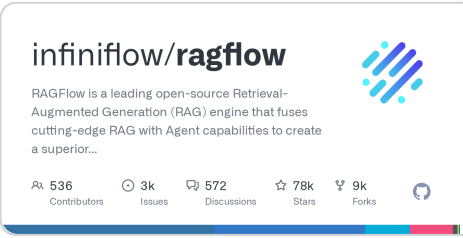
可参考，用于模拟测试部分

https://github.com/khoj-ai/khoj

GitHub - khoj-ai/khoj: Your AI second brain. Self-hostable. Get answers from the web or your docs. B

Your AI second brain. Self-hostable. Get answers from the web or your docs. Bui···

可参考文件管理与解析，搜索逻辑与LlamaIndex部分的索引/查询/评估



https://github.com/infiniflow/ragflow

GitHub - infiniflow/ragflow: RAGFlow is a leading open-source Retrieval-Augmented Generation (RAG) e

RAGFlow is a leading open-source Retrieval-Augmented Generation (RAG) engin···

文档解析动画与交互部分可参考，RRF前期不用管，后期可引入

https://github.com/Arize-AI/phoenix

GitHub - Arize-ai/phoenix: AI Observability & Evaluation

AI Observability & Evaluation. Contribute to Arize-ai/phoenix development by creating an account on GitHub.

网关层统一监督统计

技术选型

- **核心流：** `原始文档` -> `MinerU解析` -> `LLM重构编译` -> `Markdown Wiki` -> `全文索引(BM25)` -> `智能问答`

详细技术选型清单

1.1 文档处理层（The Alchemist）

- **解析引擎：MinerU (Magic-pdf)**
  - *理由*：国产最强，完美处理PDF中的复杂表格、分栏和公式，输出高质量Markdown。
- **编译模型：DeepSeek-V3**
  - *理由*：逻辑极强，擅长进行长文本重构与知识点提炼。
- **格式规范：Standard Markdown + YAML Frontmatter**
  - *规范*：强制要求AI在编译Wiki时生成标准元数据（标题、标签、别名、来源）。

1.2 数据与检索引擎（The Vault & Engine）

- **数据库：SQLite (开启 FTS5 插件)**
  - *理由*：**不引入向量数据库。** SQLite FTS5 支持原生的 **BM25 算法**，处理万级以内的 Wiki 页面性能极佳，且无需配置服务器，一个文件即走。
- **检索分词：cppjieba / jieba**
  - *理由*：解决中文全文搜索的分词问题。
- **RAG 框架：LlamaIndex**
  - *理由*：RAG 界的“重型狙击枪”，对“Node”（Wiki页面）和“Metadata”的管理远超 LangChain。
  - 注：LangChain日后再上

1.3 后端架构（The Backbone）

- **核心框架：FastAPI (Python 3.11+)**
  - *理由*：类型安全，自动生成文档，AI 助手（Cursor/Claude）写 FastAPI 几乎零错误。
- **ORM：SQLModel**
  - *理由*：FastAPI 作者出品，结合了 Pydantic 和 SQLAlchemy，全栈开发体验极佳。
- **异步处理：TaskIQ**
  - *理由*：处理 MinerU 这种耗时的解析任务，不阻塞前端对话。
- **通信协议：SSE (Server-Sent Events)**
  - *理由*：实现流式对话输出，交互感参考 Sirchmunk。

1.4 前端交互层（The Interface）

- **框架：Next.js 14 (App Router) + TypeScript**
  - *理由*：现代 Web 标准，组件化程度高，适合多人并行。
- **UI 组件库：shadcn/ui + Tailwind CSS**
  - *理由*：**Vibe Coding 核心。** 直接“复制粘贴”现成的高级组件，无需手写 CSS。
- **编辑器：MDXEditor**

- *理由*：所见即所得的 Markdown 编辑，支持实时预览，符合 Wiki 协作直觉。
- **状态管理：Zustand**
  - *理由*：极其轻量，12 人团队上手即用。

核心业务逻辑实现 (MVP 阶段)

1.5 纯文本检索实现 (No Vector)

- **原理**：利用 LLM 提取用户问题中的关键词 -> 触发 SQLite FTS5 的 `MATCH` 查询 -> 依据标题和标签权重返回最匹配的 Wiki 页面。
- **Query 重写**：在搜索前加一层 LLM，将口语（“计科多少分”）转为检索关键词（“计算机科学录取分数线”）。

1.6 冲突检测逻辑

- **原理**：在 Wiki 编译阶段，将提取的“Facts”与数据库中同标题 Wiki 的“Facts”比对。
- **触发**：若数字/日期不一致，标记 `status='disputed'`，推送到原型的“审核”页面。

2. 项目架构

1. 整洁架构 (Clean Architecture) —— 解决高可维护性与 LLM 适配

- **原理**：将业务逻辑（Wiki 编译、问答逻辑）与外部技术（MinerU、DeepSeek、Postgres）解耦。
- **落地**：定义 **抽象接口（Interfaces/Ports）**。例如，定义一个 `LLMService` 接口。
  - 现在用 DeepSeek，只需写一个 `DeepSeekAdapter`。
  - 以后想换成本地的 Ollama 或 Claude，只需增加一个 Adapter，**不需要修改任何业务代码**。
- **Vibe 优势**：Cursor 在这种层级清晰的代码结构下，生成的代码关联性极强，几乎不会出现逻辑混乱。

2. 轻量级 CQRS —— 解决数据流管理

- **Command（写路径）**：文件上传 -> MinerU 解析 -> LLM 编译 -> 生成 Wiki。这是一个**高延迟、重量级**的过程，必须走异步任务队列（TaskIQ/Redis）。
- **Query（读路径）**：用户提问 -> 检索 -> 生成答案。这是一个**低延迟、流式**的过程。
- **落地**：在数据库层面，索引表单独优化；在代码层面，将“编译逻辑”和“问答逻辑”完全分开。

3. 多租户隔离 (Multi-tenancy)

- **策略**：采用 **“字段隔离 (Row-Level Security)”** 模式。
- **落地**：
  - 所有数据库表必带 `workspace_id`。

- **核心技术**：使用 Python 的 `ContextVars`。在网关层识别租户后，将 `workspace_id` 注入全局上下文。
- **效果**：后续任何 Pod 的成员写 SQL 查询时，Repository 层自动带上 `WHERE workspace_id = context.get()`。

4. 外部工具调用与生命周期管理 (Agentic Workflow)

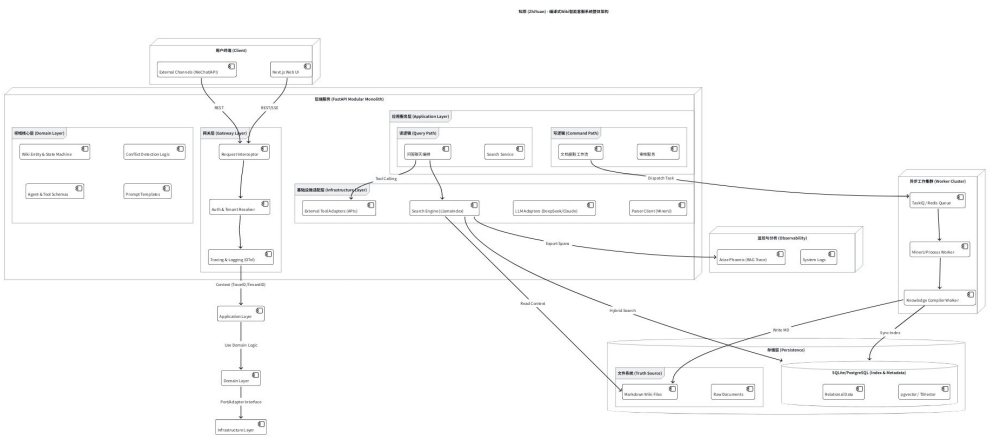
- **选型：LlamaIndex Agent + 插件化设计。**
- **落地**：将外部辅助工具封装为标准 `Tools`。
- **生命周期**：每一个 Wiki 页面都有状态机（`Draft` -> `Review` -> `Published` -> `Archived`）。在应用层统一管理状态流转。

---

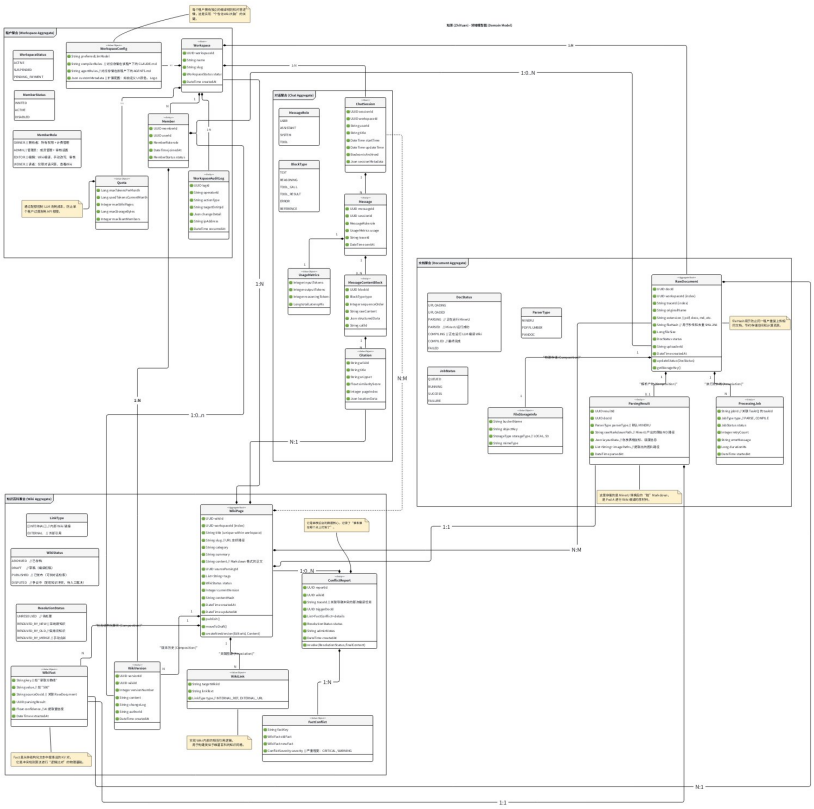
5. 全链路追踪 (Observability) —— 解决调试难题

- **选型：OpenTelemetry (OTel) + Arize Phoenix。**
- **落地**：
  - **TraceID**：在网关生成请求唯一 ID，通过 Header 传给后端，再传给异步 Worker。
  - **监控内容**：记录每一个步骤的耗时（MinerU 解析了多久？检索耗时多少？LLM 首字延迟多少？）。
  - **效果**：出现问题时，Pod D 只需看一眼 Phoenix 面板，就能准确定位是哪一个环节、哪一个 Pod 的代码出了问题。

架构图



领域模型图


聚合根 (Aggregate Root)
Workspace
Member
Quota
WorkspaceBilling
聚合根 (Aggregate Root)
Conversation
Message
聚合根 (Aggregate Root)
Document
Chunk
聚合根 (Aggregate Root)
KnowledgeBase
Workflow
WorkflowVar
WorkflowRun
AutoLoRa
1:N
1:1
1:0..N
N:M
N:1